

M19 / PROTECTED ANSWERS

M19 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
Worked SQL interview answers	5
SQL 1 - reasoning and result	5
SQL 2 - reasoning and result	6
SQL 3 - reasoning and result	6
SQL 4 - reasoning and result	7
SQL 5 - reasoning and result	7
SQL 6 - reasoning and result	7
Worked Python interview answers	8
A worked junior system-design answer	9
A truthful project walkthrough	9

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a saved test attempt. These are concrete acceptable answer shapes, not the only valid wording.

Q1 | Acceptable solved answer

Example bullet: “Built a replay-safe batch pipeline over orders/customer/product fixtures; preserved immutable raw evidence, quarantined invalid rows, reconciled source-to-target counts, and proved failure recovery.” Proof: repository run log + expected controls + failure/recovery evidence. Repeat for analytics-engineering and lakehouse projects. Never invent production users/percentages.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q2 | Acceptable solved answer

Check problem/architecture/grain/run/expected controls/failure-recovery/limitations in two minutes. Unpin a repo if its setup is broken, it is tutorial-copy without independent evidence, it exposes secrets, or you cannot defend its design.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q3 | Acceptable solved answer

Headline example: “Data Engineering Portfolio | Python • SQL • Spark • Airflow • Delta/Fabric | Reliable data-pipeline projects.” About opening: “I am transitioning into data engineering through evidence-backed projects in batch pipelines, analytics engineering, lakehouse processing and production reliability.” Do not claim a current DE job if you do not hold one.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q4 | Acceptable solved answer

“B.Tech CSE mandatory” = hard gate when no equivalent path: skip. “CS preferred” = preference: apply if core requirements/evidence fit. “Bachelor degree or equivalent practical experience” = explicit alternative route: apply if evidence supports the skills.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q5 | Acceptable solved answer

Filter PAID; rank within customer by updated_at descending. If equal timestamps can represent conflicting latest candidates, use RANK to surface them or state a deterministic secondary key such as order_id/version when business rules permit. Validate one row/customer only after tie policy is explicit.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q6 | Acceptable solved answer

Use a set. Scan arrival order; return first ID already in set, otherwise add it. $O(n)$ time average, $O(n)$ space. Empty input returns None.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q7 | Acceptable solved answer

Freeze progress; preserve source/sink/checkpoint evidence; diff source vs sink IDs; inspect rejection/quarantine and write/commit order; identify last safe durable state; repair commit ownership; replay only unsafe range; reconcile 5000 = trusted + quarantined.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q8 | Acceptable solved answer

Business need -> source/grain -> architecture -> one hard decision/alternative -> failure/recovery -> validation/reconciliation -> limitation/next step. Name your own contribution and evidence.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q9 | Acceptable solved answer

Source DB/API -> immutable raw -> 15-min micro-batch validation/transform -> trusted lakehouse/warehouse marts -> dashboard. State checkpoint/replay and reconciliation. Trade-offs: fresher data increases orchestration/compute; micro-batch simpler/cheaper than streaming when 15 minutes is sufficient.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q10 | Acceptable solved answer

Use a real course-project failure. Situation and Task are brief. Action explains reproduction, evidence, change and tests. Result states the verified outcome. Reflection names what changed in your engineering approach. Do not pretend it happened at an employer.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q11 | Acceptable solved answer

Vague: "It failed so I fixed it." Clear: "The consumer lost valid events because the sink exception was caught as bad data and the offset was committed. A controlled sink failure reproduced it; I separated infrastructure exceptions from poison records and moved commit after durable write. The failed valid event then remained replayable."

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q12 | Acceptable solved answer

First hypothesis: targeting/resume/profile mismatch, not "learn more tools." Metric: screen rate among hard-gate-passing targeted applications next week. Repair one thing - e.g., evidence-backed resume bullets/project links - and compare.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q13 | Acceptable solved answer

Clarify input/output -> tiny example -> brute-force baseline -> choose array/set/hashmap pattern -> code -> edge tests -> complexity. Timing is optional after a correct saved untimed attempt.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Q14 | Acceptable solved answer

SQL intermediate grain one row/customer. Reconcile paid-order count and total paid amount; after Python derived flags, row count and total must stay unchanged. Timestamp ties use a documented tiebreaker or are surfaced rather than silently chosen.

Self-check: Can you explain the evidence/validation without reading this page? Then close the answer and do a changed retry.

Worked SQL interview answers

These queries were executed against practice/sql/interview_fixture.sqlite. This classroom SQLite fixture uses small exact-looking numeric examples; production money should use integer minor units or an exact decimal database type. Save your first attempt before using these answers.

SQL 1 - reasoning and result

Filter orders to PAID before grouping by the customer region. EAST is 360.00 and WEST is 260.00, which reconcile to 620.00. Grouping by customer would answer a different question. An INNER JOIN is appropriate here because only regions with paid orders are requested.

```
SELECT c.region, ROUND(SUM(o.amount), 2) paid_revenue
FROM orders o
JOIN customers c USING(customer_id)
WHERE o.status='PAID'
GROUP BY c.region
ORDER BY c.region;
```

region	paid_revenue
EAST	360.0
WEST	260.0

SQL 2 - reasoning and result

NOT EXISTS asks whether a matching paid order exists, without multiplying the outer customer row. C4 is the only result. Testing status != PAID would be wrong because a customer can have both paid and unpaid orders.

```
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS
  (SELECT 1
   FROM orders o
   WHERE o.customer_id=c.customer_id
        AND o.status='PAID')
ORDER BY c.customer_id;
```

customer_id
C4

SQL 3 - reasoning and result

Filter to paid orders before ranking within each customer. RANK deliberately exposes ties: C2 has O3 and O4 at the same latest timestamp, so four rows are correct for the stated tie policy. To require exactly one row, use ROW_NUMBER with an explicit order_id tie-break and explain that choice.

```
WITH p AS
  (SELECT *, RANK() OVER(PARTITION BY customer_id
                        ORDER BY updated_at DESC) r
   FROM orders
   WHERE status='PAID')
SELECT *
FROM p
WHERE r=1
ORDER BY customer_id,order_id;
```

order_id	customer_id	updated_at	status	amount	r
O2	C1	2026-09-02T10:00:00Z	PAID	80.0	1
O3	C2	2026-09-02T09:00:00Z	PAID	220.0	1
O4	C2	2026-09-02T09:00:00Z	PAID	40.0	1
O6	C3	2026-09-04T11:00:00Z	PAID	160.0	1

SQL 4 - reasoning and result

Order by timestamp and the unique order_id to make ties deterministic. The running totals are 120, 340, 380, 460 and 620. The final running total must match the independently grouped paid total. A nonunique ordering could leave the intermediate path ambiguous.

```
SELECT order_id,updated_at,amount,SUM(amount) OVER(
                                ORDER BY updated_at,
                                order_id) running_paid
FROM orders
WHERE status='PAID'
ORDER BY updated_at,order_id;
```

order_id	updated_at	amount	running_paid
O1	2026-09-01T10:00:00Z	120.0	120.0
O3	2026-09-02T09:00:00Z	220.0	340.0
O4	2026-09-02T09:00:00Z	40.0	380.0
O2	2026-09-02T10:00:00Z	80.0	460.0
O6	2026-09-04T11:00:00Z	160.0	620.0

SQL 5 - reasoning and result

Aggregate per customer first, then rank those aggregates. C2 is top with 260.00. Ranking individual orders would find the biggest order, which is a different business question. RANK also retains all customers in a tied first place.

```
WITH x AS
  (SELECT customer_id,SUM(amount) amt
   FROM orders
   WHERE status='PAID'
   GROUP BY customer_id),
  r AS
  (SELECT *,RANK() OVER(
                                ORDER BY amt DESC) pos
   FROM x)
SELECT *
FROM r
WHERE pos=1;
```

customer_id	amt	pos
C2	260.0	1

SQL 6 - reasoning and result

Put the PAID condition in the LEFT JOIN ON clause to preserve customers with zero paid orders. COUNT(order_id) ignores the unmatched null; COUNT(*) would incorrectly give C4 a count of one. The four rows are C1 2/200, C2 2/260, C3 1/160, C4 0/0. Their amounts reconcile to 620.00.

```
SELECT c.customer_id, COUNT(o.order_id) paid_orders,
       COALESCE(SUM(o.amount), 0) paid_amount
FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id
AND o.status='PAID'
GROUP BY c.customer_id
ORDER BY c.customer_id;
```

customer_id	paid_orders	paid_amount
C1	2	200.0
C2	2	260.0
C3	1	160.0
C4	0	0

Worked Python interview answers

first_duplicate([A, B, A, C]) returns A because the second A is the first item already in the seen set. counts_by_status counts each supplied row once. first_non_repeating("swiss") returns w: s occurs three times, while w is the first character whose total count is one. These functions are linear in the number of input items under normal dictionary/set assumptions. For latest-version deduplication, do two passes: discover each ID's maximum version, then retain and compare only rows at that version. Equal maximum versions with different content raise ValueError; older disagreements do not change this interview answer. Reversing input must not change the result. reconcile_ids reports missing source IDs and unexpected target IDs; because it uses sets, it does not detect multiplicity errors. Use counts separately when duplicate frequency matters.

```
from collections import Counter
```

```
def first_duplicate(ids):
    s = set()
    for x in ids:
        if x in s:
            return x
        s.add(x)
    return None
```

```
def counts_by_status(rows):
    return dict(Counter(r["status"] for r in rows))
```

```
def first_non_repeating(text):
    c = Counter(text)
    return next((ch for ch in text if c[ch] == 1), None)
```

```
def dedupe_keep_latest(rows):
    """Keep highest version per id; reject conflicting content at that maximum.
    Older conflicting versions are irrelevant to this particular interview contract.
    """
    rows = list(rows)
    maximum = {}
    for row in rows:
```

```
    key = row["id"]
    maximum[key] = max(maximum.get(key, row["version"]), row["version"])
result = {}
for row in rows:
    key = row["id"]
    if row["version"] != maximum[key]:
        continue
    if key in result and result[key] != row:
        raise ValueError("conflicting latest version")
    result[key] = dict(row)
return result

def reconcile_ids(source_ids, target_ids):
    return {
        "missing": set(source_ids) - set(target_ids),
        "unexpected": set(target_ids) - set(source_ids),
    }
```

A worked junior system-design answer

For a small daily orders report, I would begin with scheduled batch ingestion into immutable raw storage, validate types and relationships, merge current orders using order_id and a source version, and publish a checked SQL summary. I would record a logical batch date and advance progress after successful output validation. This avoids forcing a streaming system onto a daily business requirement. My main failure case is a process stopping after the sink write but before progress commits. The repeated run must recognize the same source identity and make the same business change at most once. I would prove this with a deliberately failed run and a replay, comparing order keys and paid revenue. I would keep raw access restricted, avoid secrets in code and expose only the fields analysts need. Before scaling, I would measure arrival volume, processing duration, skew and the report's freshness requirement. If the interviewer changes the requirement to second-level freshness, I would reconsider source CDC, partitioned event delivery and streaming state. I would explicitly discuss out-of-order events, retention, replay, sink identity and operational cost. That is a requirement change, not evidence that the original daily batch design was wrong.

A truthful project walkthrough

My supplied capstone joins orders to captured payments while keeping unpaid orders visible. The fixture has 12 orders worth 1380.00, but captured payments total 1240.00 because OR009 was declined. I aggregate captures at order grain before the join and check both event identity and the business rule against duplicate captures. I deliberately inject a duplicate and prove publication stops. In a live interview, replace these sentences with your own executed evidence; do not claim a native deployment you only designed.